



分布与并行数据库

8 分布式并发控制

主讲教师：鄂金龙
中国人民大学信息学院





主要内容

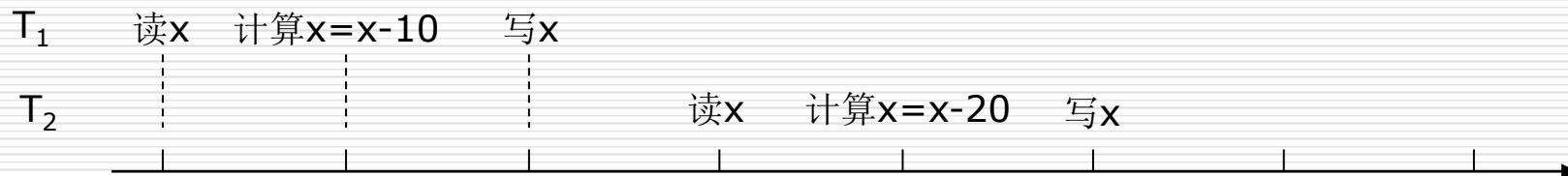
- 基本概念
- 并发控制理论基础
- 基于锁的并发控制方法
- 两段封锁协议
- 分布式数据库并发控制方法
- 分布式死锁管理
- 大数据数据库并发控制技术

基本概念

- 简单地讲，**并发就是多个事务的同时执行**，并发能够提高系统的效率，但也可能会带来三种错误（**丢失修改、不能重复读、读脏数据**）。
- 并发控制的主要目的是保证事务的一致性和隔离性，最终保证**数据的一致性**。
- 当分布事务并发执行时，并发控制既要实现分布事务的**可串行性**，又要保持事务具有良好的**并发度**，以保证系统具有良好的性能。

基本概念

□ 并发控制问题



T_1 : $R1(x)$, $O1(x)$, $W1(x)$;

T_2 : $R2(x)$, $O2(x)$, $W2(x)$;

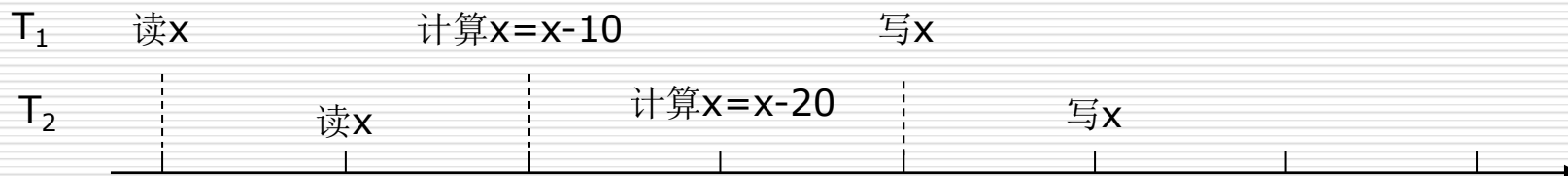
若串行执行 ($T_1 \rightarrow T_2$), x 初始值为100, 则操作序列为:

$R1(x)$, $O1(x)$, $W1(x)$, $R2(x)$, $O2(x)$, $W2(x)$,
执行结果 $x=70$ 。

基本概念

□ 并发控制问题

(1) 丢失修改错误

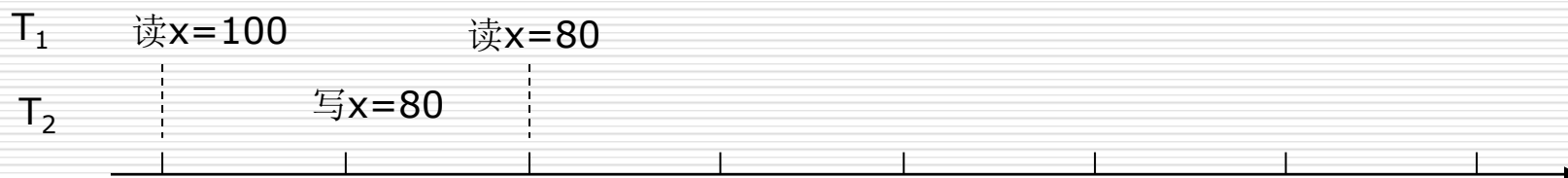


执行结果 ~~$x=80$~~

基本概念

□ 并发控制问题

(2) 不可重复读错误

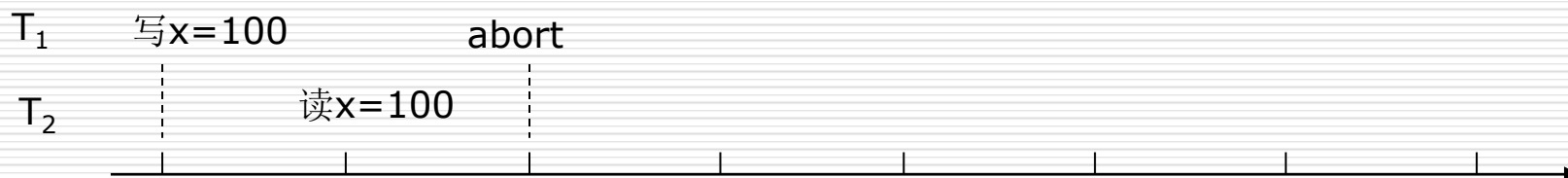


T_1 两次读取同一个数据，读取~~的~~值不同

基本概念

□ 并发控制问题

(3) 读脏数据错误



T_2 读取了 ~~T_1~~ 废弃的数据x

基本概念

□ 并发控制问题

并发控制就是利用正确的方式调度事务中所涉及的**并发操作**序列，避免造成数据的不一致性；防止一个事务的执行受到其它事务的干扰，保证事务并发执行的**可串行性**。

并发控制理论基础

□ 事务执行过程的形式化描述

通常以串行化理论来检验并发控制方法的正确性。

- 依据串行化理论，在数据库上运行的一个事务的所有操作，按其性质分为读和写两类。
- 读集与写集：一个事务 T_i 所读取数据项的集合，称为 T_i 的读集，所写的数据项的集合，称为 T_i 的写集，分别记为 $R(T_i)$ 和 $W(T_i)$ 。
- 例如：设有事务 T_1 ，完成的操作如下： $T_1: x=x+1; y=y+1$ ；则 T_1 可表示为： $T_1: R_1(x) W_1(x) R_1(y) W_1(y)$ 。

读/写集分别是：

$$R(T_1)=\{x, y\}$$

$$W(T_1)=\{x, y\}$$

并发控制理论基础

□ 事务执行过程的形式化描述

- **历程：** 在一个数据库上，各个事务所执行的操作组成的序列，称为事务的历程，记为 H ，有时也称为调度，记录了各事务的操作顺序。
- **串行历程与并发历程：** 对于一个历程上的任何两个事务： T_i 和 T_j ，如果 T_i 的最后一个操作在 T_j 的第一个操作之前完成，或反之， T_j 的最后一个操作在 T_i 第一个操作之前完成，称这样的历程为**串行历程**。否则，称为**并发历程**。
- **期望：** 事务历程中的各个事务是并发的，但同时它们的执行结果又等价于一个串行历程，即事务并发历程是可串行化的。

并发控制理论基础

□ 事务执行过程的形式化描述

例如：

有事务 T_1 和 T_2 ：

T_1 : $R_1(x) R_1(y) W_1(x) W_1(y)$

T_2 : $R_2(x) W_2(y)$

历程 H_1 和 H_2 ，分别为：

H_1 : $R_1(x) R_1(y) W_1(x) W_1(y) R_2(x) W_2(y)$

H_2 : $R_1(x) R_2(x) R_1(y) W_1(x) W_1(y) W_2(y)$

可见， H_1 为串行历程， H_2 为并行历程。

并发控制理论基础

□ 集中式数据库的可串行化问题

- 无论在集中式数据库系统中，还是在分布式数据库系统中，并发调度都要解决并发事务对数据库的冲突操作问题，使冲突操作串行执行，非冲突操作并发执行
- 在分布式数据库系统中，事务是由分解为各个场地上的子事务的执行实现的。因此，分布式事务之间的冲突操作，就转化为了同一场地上的子事务之间的冲突操作，分布式事务的可串行性调度也转化为了子事务的可串行性调度问题

并发控制理论基础

□ 集中式数据库的可串行化问题

- **可串行化**：在集中式数据库系统中的一个历程H，如果等价于一个串行历程，则称历程H是可串行化的。由可串行化历程H所决定的事务执行顺序，记为SR(H)
- 事务的**执行顺序**：分别属于两个事务的两个操作 O_i 和 O_j ，如果它们操作同一个数据项，且至少其中一个操作作为**写操作**，则 O_i 和 O_j 这两个操作是**冲突**的。例如： $R_1(x)$ 和 $W_2(x)$ ， $W_1(x)$ 和 $W_2(x)$ 。
- 在一个串行历程中，用符号“ $<$ ”表示先于关系，对分别属于 T_i 和 T_j 的两个冲突操作 O_i 和 O_j ，若存在 $O_i < O_j$ ，则 $T_i < T_j$

并发控制理论基础

□ 集中式数据库的可串行化问题

■ 历程等价的判别方法

定理1: 任意两个历程 H_1 和 H_2 等价的充要条件是

- ▶ 在 H_1 和 H_2 中, 每个读操作读出的数据是由相同的写操作完成的;
- ▶ 在 H_1 和 H_2 中, 每个数据项上最后的写操作是相同的

■ **定理1的引理:** 对于两个历程 H_1 和 H_2 , 如果每一对冲突操作 O_i 和 O_j , 在 H_1 中有 $O_i < O_j$, 在 H_2 中也有 $O_i < O_j$, 则 H_1 和 H_2 是等价的

并发控制理论基础

□ 集中式数据库的可串行化问题

■ 历程等价的判别方法

例: $H_1: R_1(x) R_1(y) W_1(x) W_1(y) R_2(x) W_2(x)$

$H_2: R_1(x) R_1(y) W_1(x) R_2(x) W_1(y) W_2(x)$

$H_3: R_1(x) R_1(y) R_2(x) W_1(x) W_1(y) W_2(x)$

H_1 是串行历程。判断 H_2 和 H_3 是否与 H_1 等价（是可串行化的历程）。

H1的冲突操作:

$R_1(x) < W_2(x)$

$W_1(x) < W_2(x)$

$W_1(x) < R_2(x)$

H2的冲突操作:

$R_1(x) < W_2(x)$

$W_1(x) < R_2(x)$

$W_1(x) < W_2(x)$

H3的冲突操作:

~~$R_1(x) < W_2(x)$~~

~~$R_2(x) < W_1(x)$~~

~~$W_1(x) < W_2(x)$~~

并发控制理论基础

□ 分布式事务的可串行化问题

- **局部历程**：在分布式事务执行过程中，场地 S_i 上的所有子事务的操作的执行序列，称为局部历程，用 $H(S_i)$ 表示。
- **分布式事务可串行化判定定理**：对于 n 个分布式事务 $T_1, T_2, \dots, T_n$ 在 m 个场地上 $S_1, S_2, \dots, S_m$ 上的并发执行序列，记为 E 。如果 E 是可串行化的，则必须满足以下条件：
 - ▶ 每个场地 S_i 上的**局部历程** $H(S_i)$ 是可串行化的；
 - ▶ 存在 E 的一个总序，使得在**总序**中，如果有 $T_i < T_j$ ，则在各局部历程中必须有 $T_i < T_j$ 。

并发控制理论基础

□ 分布式事务的可串行化问题

分布式事务可串行化判定定理的**引理**:

设 $T_1, T_2, \dots, T_n$ 是 n 个分布式事务, E 是这组事务在 m 个场地上的并发执行序列, $H(S_1), H(S_2), \dots, H(S_m)$ 是在这些场地上事务的局部历程, 如果 E 是可串行化的, 则必须存在一个总序, 使得 T_i 和 T_j 中的任意两个冲突操作 O_i 和 O_j , 如果在 $H(S_1), H(S_2), \dots, H(S_m)$ 中有 $O_i < O_j$, 当且仅当在总序中也有 $T_i < T_j$ 。

基于锁的并发控制方法

- **基本思想**：事务在对某一**数据项操作之前**，必须先申请对该数据项**加锁**，申请成功后才可以对该数据项进行操作。如果该数据项已被其它事务加了不相容的锁，那么后申请使用数据的事务必须等待，直到该数据项被解锁为止。
- 锁分两种类型：排它锁(exclusive)和共享锁(shared)
- 锁的相容性：

	读锁	写锁
读锁	共享	排斥
写锁	排斥	排斥

基于锁的并发控制方法

□ 封锁规则

- 事务在执行过程中需对其访问的数据项进行**加锁**，访问结束要及时**释放**其对数据项加的锁，以便供其它事务访问，保证多个事务正确地并发执行。
- 具体封锁规则为：
 - 事务T在对数据项A进行读/写操作之前，必须对数据项A施加读/写锁，访问后立即释放已申请的锁；
 - 如果事务T申请不到希望的锁，事务T需等待，直到申请到所需要的锁之后，方可继续执行。

基于锁的并发控制方法

□ 锁的粒度

- 封锁数据对象的单位称锁的粒度，指**被封锁的数据对象的大小**。锁的粒度也称锁的大小。系统根据自己的实际情况确定锁的粒度，锁的粒度可以是关系的**属性**（或字段）、关系的**元组**（或记录）、关系（也称文件）或整个数据库等。
- 系统的并发度与锁粒度成反比。锁粒度越大，系统的开销越小，但降低了并发度。

□ 锁的种类

- 显式锁，用户加封锁命令实现
- 隐式锁，系统自动加锁实现

粒度	开销	并发度
小	大	高
大	小	低

两段封锁协议

- 两段封锁协议(2PL)是数据库系统中解决并发控制的重要方法之一，保证事务的**可串行性调度**。
- 2PL的实现思想是将事务中的**加锁操作**和**解锁操作分两阶段完成**，要求并发执行的多个事务要在对数据操作之前进行加锁，且每个事务中的所有加锁操作要在解锁操作以前完成。
- 两段封锁协议分为：
 - **基本的两段封锁协议**
 - **严格的两段封锁协议**

两段封锁协议

□ 基本的两段封锁协议

基本的两段封锁协议分加锁阶段和解锁阶段。

阶段1：加锁阶段

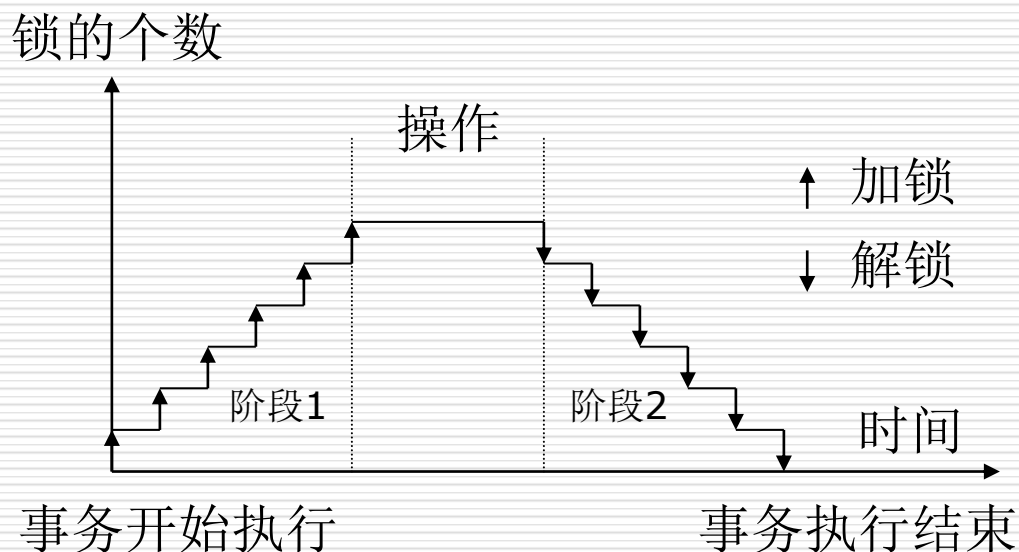
- 事务在读写一个数据项之前，必须对其加锁；
- 如果该数据项被其它使用者已加上不相容的锁，则必须等待。

阶段2：解锁阶段

事务在释放锁之后，不允许再申请其它锁；

两段封锁协议

□ 基本的两段封锁协议

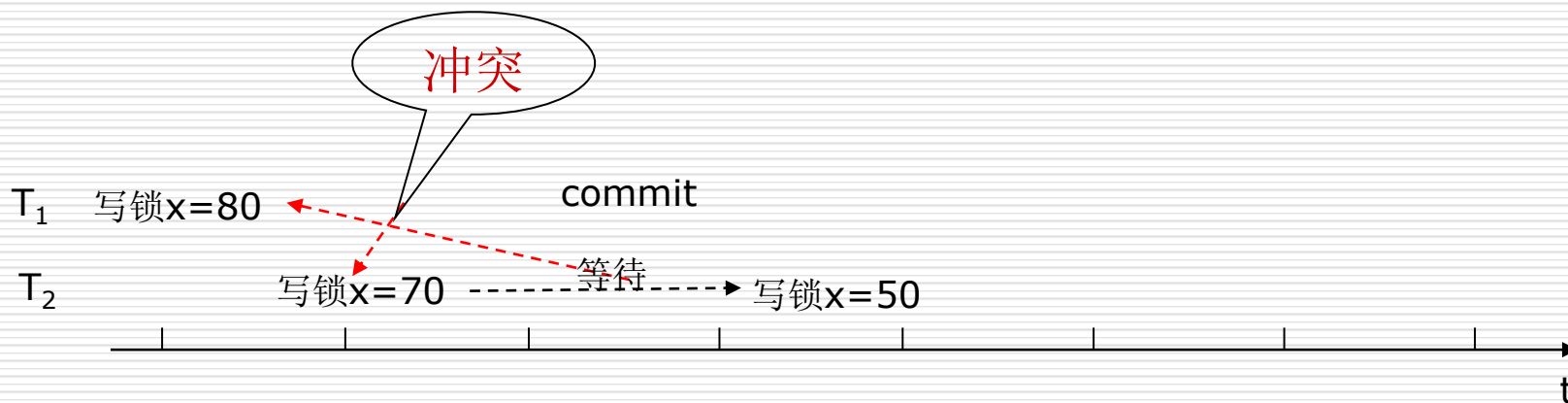


两阶段封锁协议是否可以解决并发事务带来的3个错误？

两段封锁协议

□ 基本的两段封锁协议

丢失修改错误

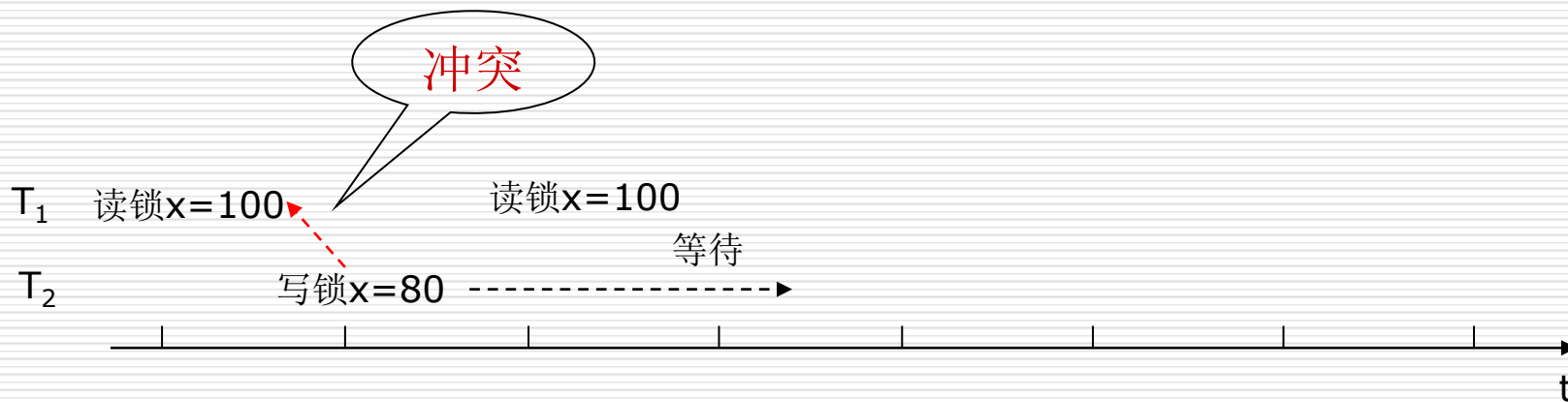


$X=100$: 当事务 T_2 : $x-30$ 申请写锁时, 不能申请成功, 必须等待, 当事务 T_1 : $x-20$ 提交数据项 x , 放锁后, T_2 才能申请到锁, 读到 $x=80$, 结果为 50, 避免了丢失修改错误。

两段封锁协议

□ 基本的两段封锁协议

不能重复读错误

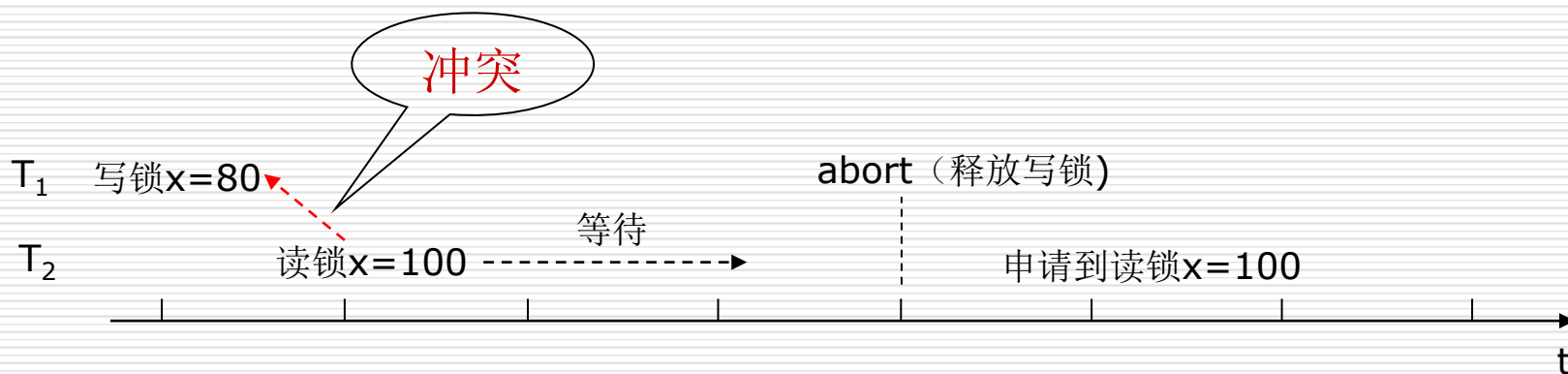


当事务 T_2 申请写锁时，不能申请成功，必须等待，当事务 T_1 再次读数据项 x 时，读取的 x 值与第一次读到的值相同，避免了不能重复读错误。

两段封锁协议

□ 基本的两段封锁协议

读脏数据错误



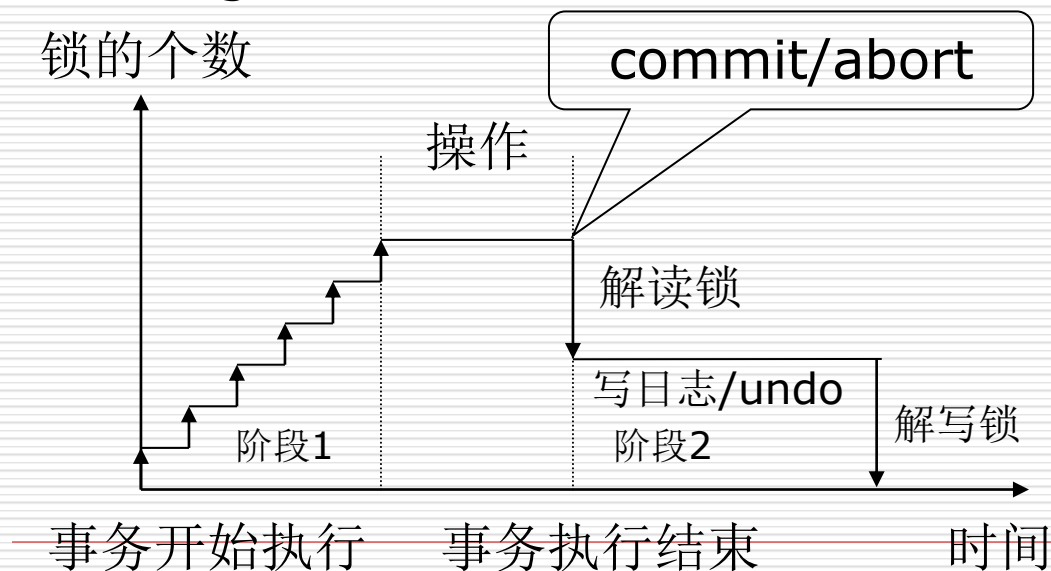
事务 T_1 申请写锁后，写 $x=80$ 。此时事务 T_2 申请读锁失败，必须等待。当事务 T_1 产生故障中断时，废弃事务 T_1 的执行，即执行undo，使 x 恢复为原值100，并释放对 x 加的写锁。此时事务 T_2 申请到读锁，读取 x 值， x 值为原值100，而不是事务 T_1 的脏数据80。

两段封锁协议

□ 严格的两段封锁协议

严格的两段封锁协议与基本两段封锁协议内容基本上是一致的，只是**解锁时刻不同**，是在**事务结束时才启动解锁**，保证了事务所更新数据的永久性。采用两段封锁协议（2PL）的事务执行过程为：

begin_transaction → 加锁 → 操作 → commit/abort → 解锁。



(1) 对commit的处理

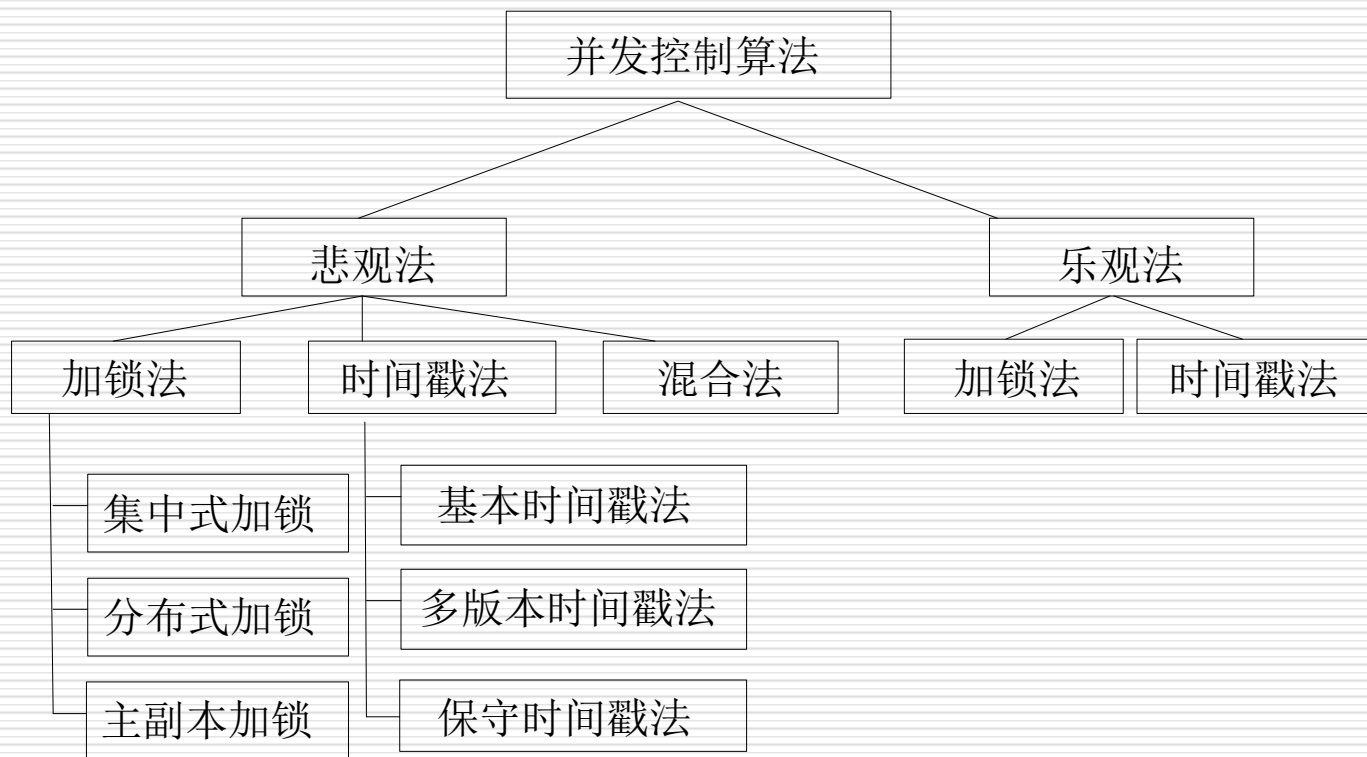
- 释放读锁;
- 写日志;
- 释放写锁;

(2) 对abort的处理

- 释放读锁;
- undo处理;
- 释放写锁;

分布式数据库的并发控制

基于锁的并发控制方法、时间戳方法、乐观方法等

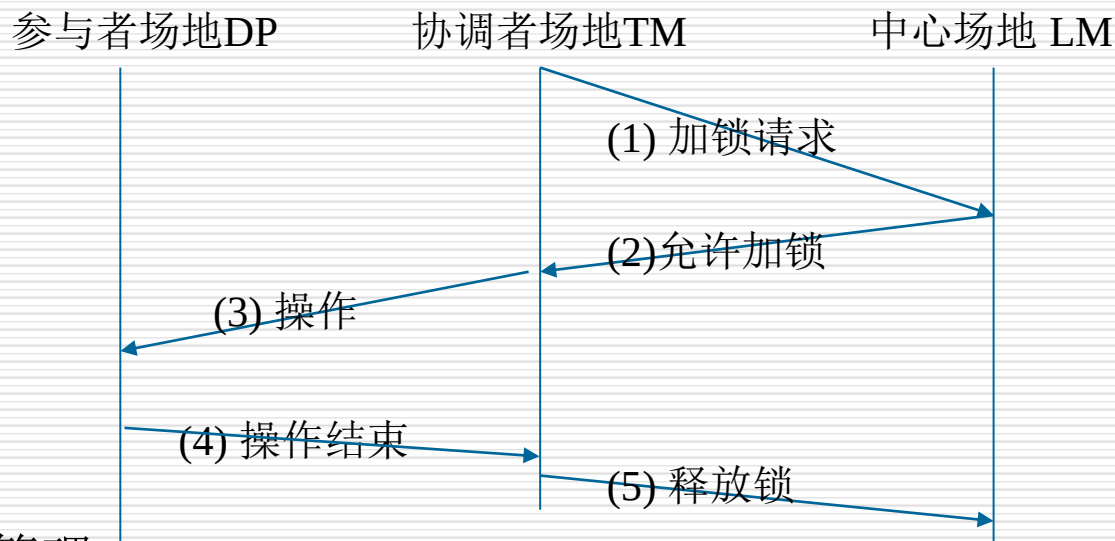


分布式数据库的并发控制

□ 基于锁的并发控制方法的实现

(1) 集中式的实现方法

集中式实现方法是在分布库中设立一个**2PL调度器**，所有封锁请求均由该调度器完成，每个场地的事务管理器都和该调度器通信。



优点：实现简单，方便管理；

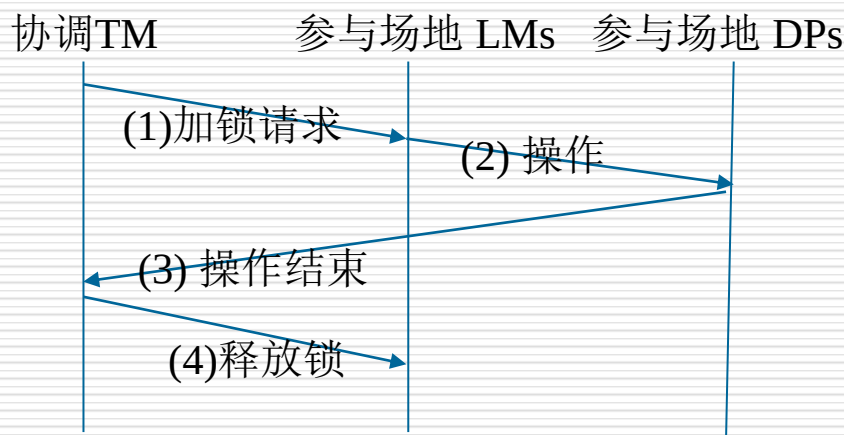
缺点：中心场地负载重，单点失效；

分布式数据库的并发控制

□ 基于锁的并发控制方法的实现

(2) 分布式的实现方法

分布式实现方法是在每个场地上都有一个2PL调度器，负责处理本场地上的封锁请求。



优点：避免了集中式封锁方法的缺点；

缺点：实现复杂，锁管理复杂；

分布式数据库的并发控制

□ 基于锁的并发控制方法的实现

(3) 对复制数据的封锁方法

基于特定副本的封锁方法

主副本法：规定一个数据项的多个副本中的一个作为主副本，所有对该数据项的封锁请求都施加在主副本上。

主场地法：规定保存副本的某个场地为主场地，所有封锁请求都施加在主场地上的数据项副本上。

带有后备场地的主场地法

基于投票的封锁方法

读写全法：事务对数据项加读锁时，只封锁其中一个副本；若加写锁，必须封锁所有副本。

多数副本法是：无论读锁还是写锁申请，必须封锁数据项一半以上的副本。申请成功后，若为读锁，读取一个副本的值；若为写锁，需向所有副本发送新值。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

基于锁的并发控制是通过事务的互斥来维护并发事务的可串行化的；

基于时间戳的方法是通过事务的优先级来实现并发事务的可串行化。

事务的优先级通过时间戳来实现，事务管理器为每个事务 T_i 在其产生时都设置时间戳， $TS(T_i)$ 。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(1) 基本概念

- **时间戳(TimeStamp)**是基于事务启动时间点，由系统赋予该事务的全局唯一标识。同一事务管理器产生的时间戳是单调增加的，可以通过**时间戳来区分事务**。
- 设置时间戳的方法主要有以下几种：一种是使用全局单调递增计数器，但是全局的计数器比较难维护。
- 一般每个场地都有自己的单调递增的计数器,为了保证唯一性，**时间戳是一个二元组<本地计数器值，场地id>**，要保持各局部时钟的同步，则才能实现全局有序。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(1) 基本概念

通常，遵循如下规则设置时间戳：

- ① 每个事务在启动场地赋予一个全局唯一标识（时间戳）；
- ② 事务的每个读操作或写操作都带有本事务的时间戳；
- ③ 数据库中的每个数据项都记录有对其进行读操作和写操作的
最大时间戳。设数据项为X，X的读/写操作的最大时间戳分
别标记为 $rts(X)$ 和 $wts(X)$ ；
- ④ 如果事务被重新启动，则其被赋予新的时间戳。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(1) 基本概念

- 基于时间戳的并发控制方法的**思想**：给每个事务赋予一个唯一时间戳，根据时间戳对事务的操作执行顺序进行排序，则**事务按时间戳顺序串行执行**。如果发生冲突，撤销一个事务并重新启动，同时为重新启动事务赋予新的时间戳。
- 时间戳排序(TimeStamp Ordering)规则：分别属于事务 T_i 和事务 T_k 的两个冲突操作 O_i^j 和 O_k^l ， O_i^j 在 O_k^l 之前执行当且仅当 $ts(T_i) < ts(T_k)$ ，称 T_i 是较老的事务， T_k 是较新的事务。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(1) 基本概念

■ 基于时间戳的事务并发控制的调度方法

① 设 ts 是对数据 X 进行读操作的时间戳，若 $ts < rts(X)$ ，则拒绝该操作，重新启动该事务并赋予新的时间戳；否则执行读操作， $rts(X) = ts$ 。

② 设 ts 是对数据 X 进行写操作的时间戳，若 $ts < rts(X)$ 或 $ts < wts(X)$ ，则拒绝该操作，重新启动该事务并赋予新的时间戳；否则，执行写操作， $wts(X) = ts$ 。

■ 基于时间戳的并发控制方法不会出现死锁，任何事务不会被阻塞。若某个事务的某个操作不能执行，则事务重新启动。但是，基于时间戳的并发控制方法避免死锁是以重新启动为代价的。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(2) 基本的时间戳排序(Basic TO)算法

事务管理器TM对每个事务设置时间戳并且将其附着在事务所包含的每个数据库操作上。

调度器SC负责跟踪读写时间戳并进行序列检查。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(2) 基本的时间戳排序(Basic TO)算法

```
while(1){ -----事务管理器(TM)端算法-----  
    监听各个场地的消息;  
    如果是数据库操作信息, 那么判断操作类型:  
        --开始事务, 则将场地集合清空, 并设置事务时间戳ts(T);  
        --读, 选择执行场地加入场地集合, 并将操作信息和事务时间戳发送给目标场地调度器;  
        --写, 存储数据的场地加入场地集合, 并将操作类型和事务时间戳发送给目标场地调度器;  
        --commit/abort, 则将操作发送给场地的调度器;  
    如果是调度器信息, 那么判断信息内容:  
        --拒绝, 则将"Abort T"添加到消息中, 发送给场地中的调度器, 重新设置时间戳;  
        --数据处理信息, 则根据不同处理类型执行不同操作:  
            ---读, 则返回给应用读的结果;  
            ---写, 则通知用户写完成;  
            ---commit/abort, 则通知应用程序提交/废弃完成;  
}
```

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(2) 基本的时间戳排序(Basic TO)算法

While(1){-----调度器(SC)端算法-----}

监听TM发送的数据库操作消息;

初始化参数, 保存读写时间戳, 判断消息内容, 执行相应的操作:

---读操作, 则比较时间戳, 如果 $ts(T) > rts(x)$, 则将数据操作信息发送给本场地DP, 并且将 $rts(x)$ 修改为 $ts(T)$; 如果 $ts(T) < rts(x)$ 则拒绝该事务的操作, 并将拒绝消息发送给TM;

---写操作, 若 $(ts(T) > rts(x) \& \& ts(T) > wts(x))$, 则发送处理操作给本场地DP, 并且将 $rts(x)$ 和 $wts(x)$ 都修改为 $ts(T)$; 否则, 拒绝行该操作, 并通知TM。

---提交, 则发送数据操作给本场地DP;

---废弃, 则对于所有被事务T访问过的数据恢复 $rts(X)$ 和 $wts(X)$ 为其初始值, 发送数据操作给本场地DP。

}

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

- 基本的TO算法不会产生死锁，但会带来过多的重启动，导致系统性能下降。
- 保守的TO算法的思想：通过减少事务重启的数量减少系统的开销。
- 基本TO算法试图当接收到一个操作时就立即执行该操作，而保守算法是希望尽可能延迟每个操作，直到保证调度器中没有时间戳更小的操作。如果这个条件可以保证的话，那么调度器将不会拒绝操作，但是这种延迟可能会带来死锁。

分布式数据库的并发控制

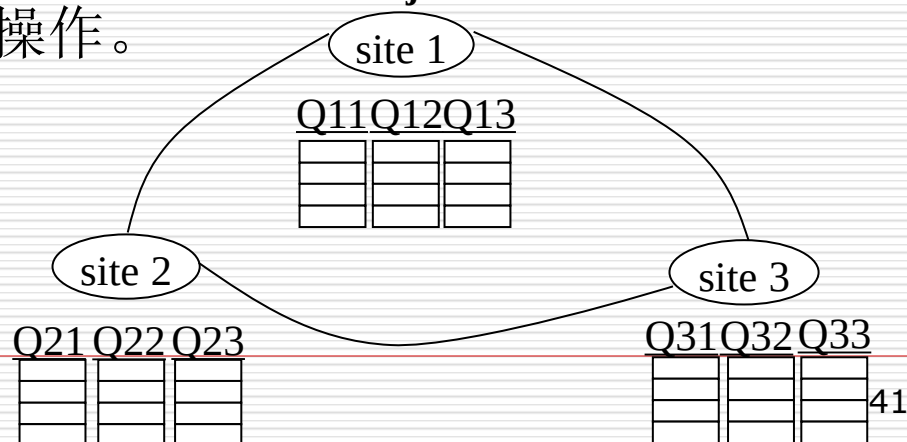
□ 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

基本思想：每个事务的操作都被缓存起来并建立起有序的队列，然后按照顺序执行操作。

具体做法：在场地site i ，每个调度器 i 有一个队列 Q_{ij} ，用于对应场地 j 的TM，简称 TM_j 。一个来自于 TM_j 的操作按照时间戳递增的顺序到达site i ，并放置于 Q_{ij} 中。调度器 i 按照这个顺序执行所有队列的操作。

例子：存在3个场地，每个场地上的调度器都为所有场地的TM建立一个事务操作的队列。



分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

对于读操作R, 若在Site i有缓冲的写, 使得

$$TS(R) > TS(W)$$

即申请的读新于已有的写, 则R被缓冲, 直到所有这样的写完成。

对于写操作W, 若在Site i有缓冲的读, 使得

$$TS(W) > TS(R)$$

或有缓冲的写W', 使得

$$TS(W) > TS(W')$$

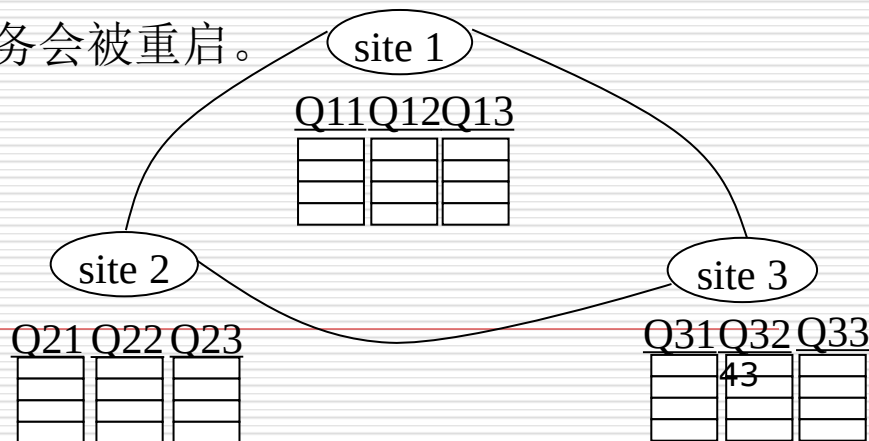
则W被缓冲, 直到所有这些操作被执行完毕。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

保守时间戳算法可以减少大量的重启，但是当队列是空的时候，还是会出现重启的现象。例如：若场地i上对场地j的队列 Q_{ij} 是空的，场地i的调度器将选择一个最小时间戳的操作发送给数据处理器，可是，场地j可能已经发送给i一个带有更小时间戳的操作，当操作到达场地i的时候，将会被拒绝，来自场地j的事务会被重启。



分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

■ 减少重新启动的另外一种方法，具体做法：

修改 x 上时间戳，对数据项 x

➤ RTM(x) 有一组值，记录了所有 T_i 对 x 的读时，
即 $RTM(T_i, x)$

➤ WTM(x) 有一组值，记录了所有 T_i 对 x 的写时，
新值也被记录，即 $WTM(T_i, x, v_i)$

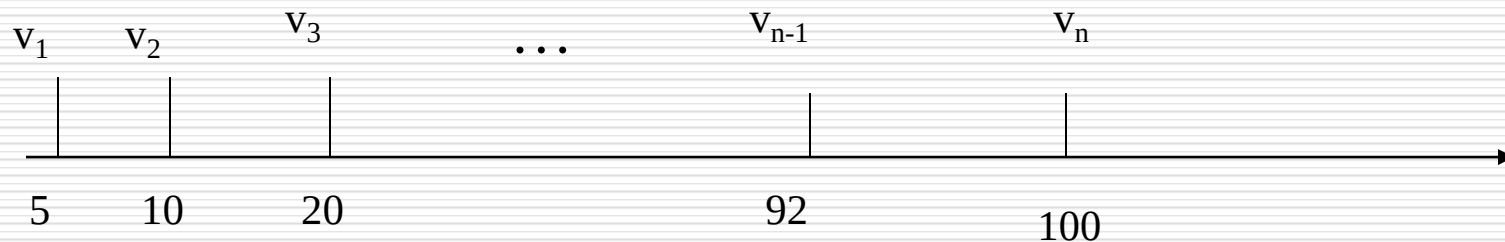
分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

■ 当要对x读时，读操作永不拒绝。假定读操作的时间戳是TS，于是在 $WTM(T_i, x, v_i)$ 中找所有小于TS的 $\{WTM(T_i, x, v_i)\}$ 中的最大 $WTM(T_j, x, v_j)$ ，将 v_j 给读的事务。

■ 例如：



若读 $TS(R_i)=95$ ，则读 $\langle 92, v_{n-1} \rangle$ 的值。

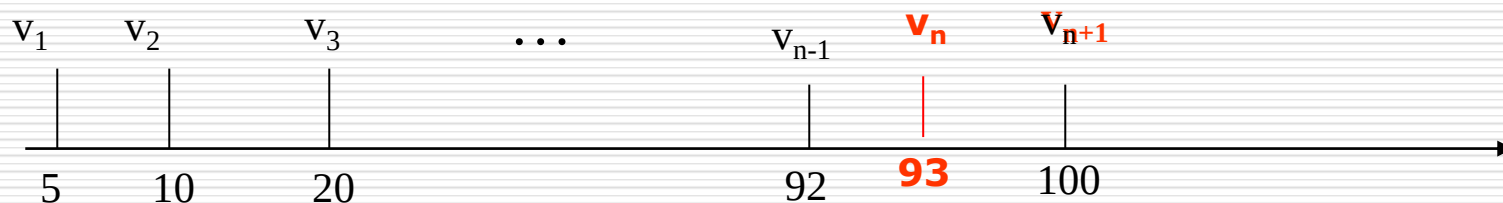
分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

- 当要对x写时，一般不拒绝。只记录下新的WTM($TS(T_k)$, x , v_k) 即可。若存在一个拥有比 $TS(T_k)$ 大的时间戳的事务(更加新的事务)已经对x完成了读，那么拒绝 $TS(T_k)$ 的写。

- 例如：



$TS(R_i)=95$, 读走 $\langle 92, v_{n-1} \rangle$ 。此时，若写 $TS(W_k)=93$ 请求写，那么拒绝 T_k 的写。

否则， T_i 事务应该读走 $\langle 93, v_n \rangle$ ，也就是说刚才 T_i 读错了数据。

分布式数据库的并发控制

□ 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

- 多版本TO算法是一种以牺牲存储空间来换取尽量少的事务重启。
- 如果采用多版本TO的并发控制方法，为了节省空间，应经常对数据库的多个版本进行清理。
- 当分布式DBMS确定不再接收需要访问被清除版本的事务时，执行清除操作。

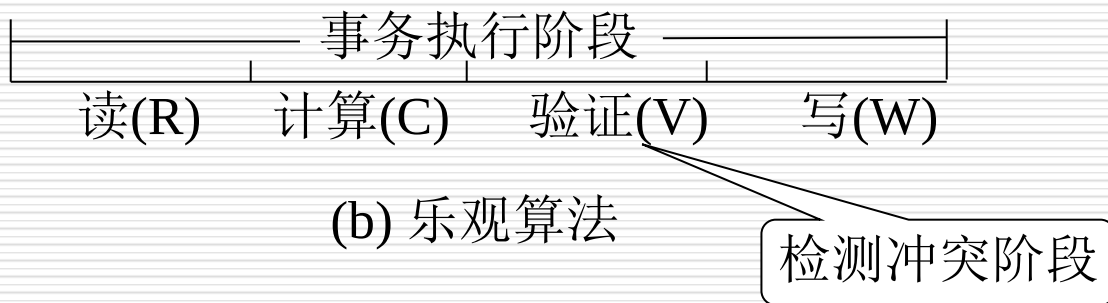
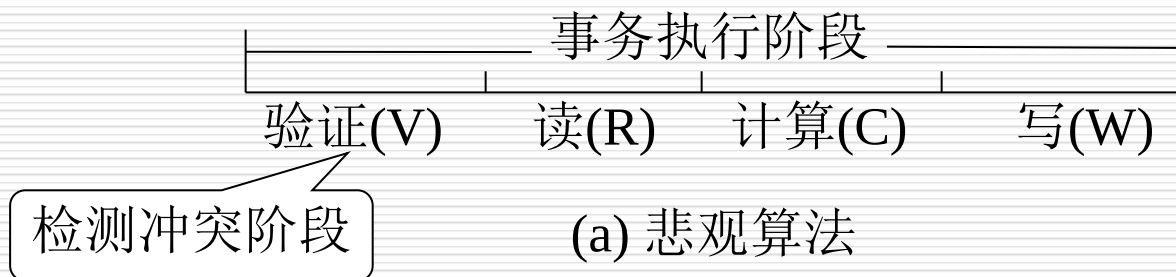
分布式数据库的并发控制

□ 乐观的并发控制算法

- 悲观算法假设冲突经常发生，而乐观算法等到写阶段开始时，才进行冲突验证，也就是说延迟验证阶段直到执行写操作之前。
- 提交给乐观调度器的操作不会被延迟，每个事务的读、计算和写操作可以自由处理，不需要更新实际的数据库。
- 每个事务在初始时都在本地更新副本，验证阶段检查这些更新是否能够维护数据库的一致性，如果结果是肯定的，就把这些更新写到实际的数据库中。否则，该事务就废弃并被重启。

分布式数据库的并发控制

□ 乐观的并发控制算法



分布式数据库的并发控制

□ 乐观的并发控制算法

■ 乐观控制算法的规则：

- 时间戳只和事务相关，而与数据项无关。
- 时间戳不是在初始化时而是在开始验证时赋给事务，因为时间戳只有在验证阶段才需要，过早赋值可能会导致不必要的事务拒绝操作。
- 每个事务 T_i 又被分为许多子事务，每个子事务都可以在许多场地上执行。

分布式数据库的并发控制

□ 乐观的并发控制算法

乐观控制算法的**规则**:

令 T_{ij} 为在场地 j 上执行的 T_i 的子事务，每个场地上子事务按一定执行序列执行，直到验证阶段，时间戳被赋给事务并拷贝给它的所有子事务，则 T_{ij} 的本地验证规则为(这些规则是**互斥**的):

- **规则1:** 如果所有事务 T_k ，其中 $ts(T_k) < ts(T_{ij})$ ，在 T_{ij} 开始读操作前完成了它们的写操作阶段，那么验证成功。
实际上是多事务的串行执行。
- **规则2:** 如果任何事务 T_k ，满足 $ts(T_k) < ts(T_{ij})$ ，在 T_{ij} 读阶段完成 T_k 写操作，当 $WS(T_k) \cap RS(T_{ij}) = \Phi$ 时，验证成功。
并发执行 T_{ij} 的读和 T_k 的写，但是 T_k 的写不影响 T_{ij} 的读。
- **规则3:** 如果任何事务 T_k ，满足 $ts(T_k) < ts(T_{ij})$ ，在 T_{ij} 完成读阶段之前完成 T_k 读操作，当 $WS(T_k) \cap RS(T_{ij}) = \Phi$ 并且 $WS(T_k) \cap WS(T_{ij}) = \Phi$ 时，验证成功。
 T_k 先完成读，然后并发执行 T_{ij} 和 T_k ，如果 T_k 的写不影响 T_{ij} 的读且 T_{ij} 和 T_k 的写集交集为空，那么也不互相影响。

分布式死锁管理

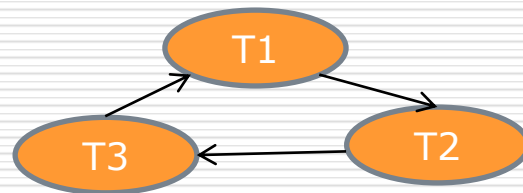
- 在使用锁的并发控制中，可能导致死锁的产生。
产生死锁原因：
 - 因为在多个事务并发执行的情况下，每个正在执行的事务已对其拥有的资源进行加锁。
 - 已经占有了部分资源的事务可能会因为其他事务对其需要的资源已经加了不相容的锁而处于等待状态。
 - 多个彼此相互等待对方释放自己需要的资源的事务就会产生死锁。
- 因此，在分布式数据库管理系统中必须设计合适的处理死锁的算法。

分布式死锁管理

□ 死锁等待图

在数据库系统中通常采用等待图方法分析死锁。等待图(Wait-For Graph, WFG)是一个表示事务之间等待关系的有向图，图中节点表示系统中的并发事务，从 T_i 指向 T_j 的有向边表示事务 T_i 等待事务 T_j 释放其对某些资源所加的锁。

当且仅当等待图中的边存在回路，即出现死锁。



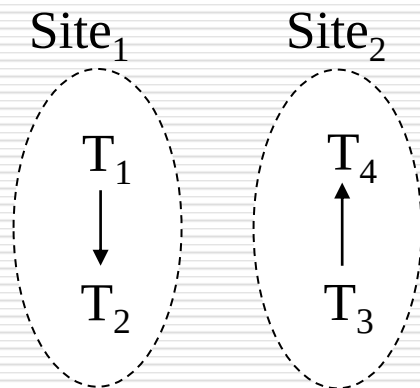
分布式死锁管理

□ 死锁等待图

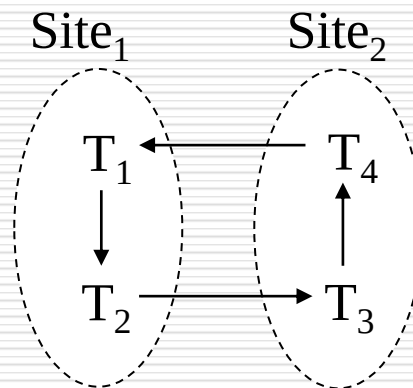
- 等待图的方法也同样适用于分布式数据库系统中。不过其检测过程比在集中式数据库更复杂，因为导致死锁产生的两个事务很可能不在同一场地上，称为全局死锁。
- 在分布式数据库的死锁检测中，除了要检测各场地事务的局部等待图(LWFG)是否存在回路，还要检测所有场地事务之间的全局等待图(GWFG)是否存在回路。
- 全局等待图由局部等待图组成，首先要得到局部等待图，然后由所有的局部等待图联合而成全局等待图。

分布式死锁管理

□ 死锁等待图



(a)局部等待图



(b)全局等待图

场地Site1和场地Site2分别有事务 T_1 、 T_2 、 T_3 、 T_4 。在分布式死锁检测中，首先要对每个场地的事务进行等待图的绘制。从(a)可知，在局部站点中并未出现回路，还需进一步判断各站点之间的事务是否存在等待的状态。将(a)中的局部等待图按照事务间的等待关系联合起来，形成图(b)。可以看到，事务 T_1 、 T_2 、 T_3 、 T_4 在全局等待图中构成了一个回路，因而产生了死锁。

分布式死锁管理

□ 死锁的检测

(1) 集中式死锁检测

- 选择一个场地执行整个系统的死锁检测程序(也称为集中死锁检测器)。
- 每个场地的锁管理器都要向该死锁检测器传输其LWFG, 从而逐渐构成GWFG。如果在GWFG中发现回路, 则认为存在死锁并采取相应的处理措施。
- 不足: 频繁的LWFG传输会带来大量的通信代价, 因此, 各个场地的锁管理器只需传输其等待图中变化部分的信息, 以减少通信量。

分布式死锁管理

□ 死锁的检测

(1) 集中式死锁检测

■ 优点：实现简单

■ 缺点：

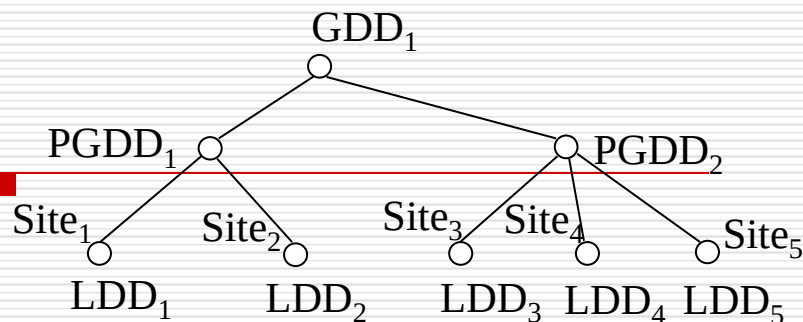
- 瓶颈、单点失效。
- 难于确定各个场地向检测器传输信息的时间间隔。间隔小，可减小死锁检测延迟，但增加数据传输代价；间隔大，可减少数据传输量，但影响死锁检测及时性。
- 通信代价大。各个场地周期地向死锁检测器场地发送消息，然而真正导致死锁产生的事务可能只涉及到少数几个场地。

分布式死锁管理

□ 死锁的检测

(2) 层次死锁检测

- 是对集中式死锁检测方法的改进，目的是减少通信量。
- 层次死锁的检测方法是建立一个死锁检测层次树，每个站点的局部死锁检测程序(LDD)作为一个叶子节点，中间层节点是部分全局死锁检测程序(PGDD)，根节点是全局死锁检测程序(GDD)。
- 每个场地首先在本地用等待图检测死锁并做出相应的处理，然后将局部等待图发送给上层节点。每个非叶子都至少包括两个下层节点，从底层至上层，部分全局死锁检测程序可以检测其所包括的下层节点的事务是否产生死锁，直到最顶层的根节点，由根节点死锁检测程序判断全局事务是否产生死锁。



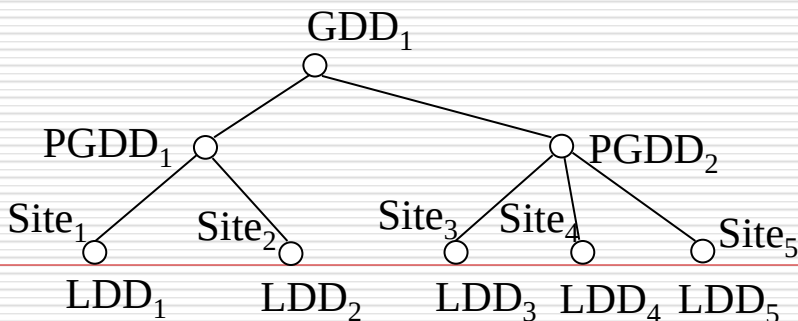
分布式死锁管理

□ 死锁的检测

(2) 层次死锁检测

■ 例如：

场地1至场地5的死锁检测由LDD₁至LDD₅完成。PGDD1负责检测场地1与场地2中的事务是否构成死锁，PGDD2负责检测场地3、场地4与场地5中的事务是否构成死锁，GDD1负责检测全局死锁。



分布式死锁管理

□ 死锁的检测

(2) 层次死锁检测

- 与单纯的集中式死锁检测方法相比，层次死锁检测方法减少了数据传输量，是使用集中式2PL并发控制算法系统中常选择的死锁检测方法。
- 层次死锁检测方法的运行性能与层次的选择至关重要，在进行场地组划分时，应考虑站点之间的物理距离以及站点上容纳的数据的相关度等。
- PGDD数目过大会增加运行成本，过少将失去其优越性。

分布式死锁管理

□ 死锁的检测

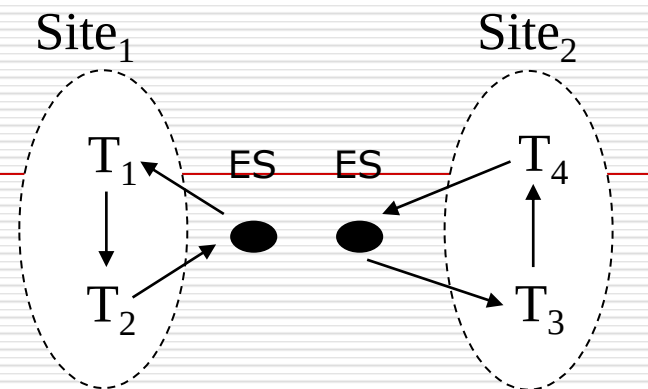
(3) 分布式死锁检测

- 检测由各个场地共同完成，每个场地都是对等的，不区分全局死锁和局部死锁的检测程序，都承担检测全局死锁的任务。
- 在检测过程中，局部WFG与其它场地的WFG相互通信，测试是否存在回路。

分布式死锁管理

□ 死锁的检测

(3) 分布式死锁检测



例如，路径下推(Path-pushing)算法，主要思想是：

- 每个场地都接受来自于其它场地的潜在的死锁回路，并将这些边加入到本地的LWFG中。
- 等待其它场地事务的WFG中的边，并与远程节点的LWFG的边相连接，从而判断是否存在死锁。
- 如果回路不包括外部边，则为局部死锁，可以被局部处理；如果回路中包括外部边，则存在潜在的分布式死锁，必须将该回路信息通知给其它场地的死锁检测程序。

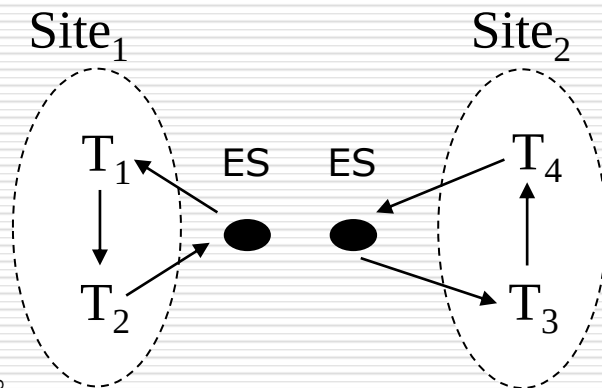
分布式死锁管理

□ 死锁的检测

(3) 分布式死锁检测

■ 例如： $T_1 \rightarrow T_2 \rightarrow Es$ 和 $T_3 \rightarrow T_4 \rightarrow Es$ 两个回路中都包括了外部结点 (External site, Es)，可以判断存在潜在的死锁回路，并且两个场地都可以检测到可能的分布式死锁。

- 每个场地都能检测出死锁回路，应该选择合适的操作场地以减少死锁检测代价。
- 思想：以沿着死锁回路正向或者反向传输信息，得到信息的场地更新它的LWFG并检查死锁。
- 如图，场地1发送它的潜在死锁信息给场地2，场地2也发送潜在死锁信息给场地1。两个场地上都将检测死锁，实际上只需要一个场地检测即可。



分布式死锁管理

□ 死锁的检测

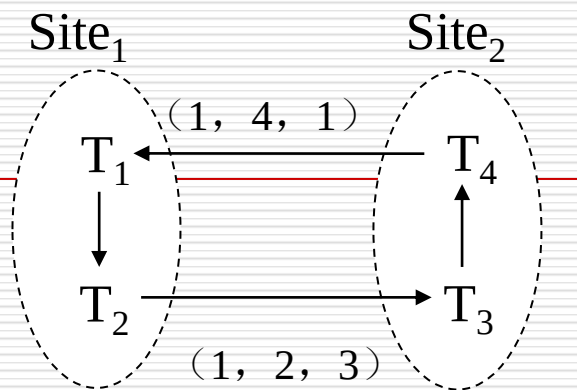
(3) 分布式死锁检测

边追踪(Edge-Chasing)算法： 沿着资源请求图上的等待边方向传递信息，并由各节点根据目前收集到的信息，判断是否出现了死锁，算法使用了LWFG来检测局部死锁，并且利用探测信息确认是否存在全局死锁。

分布式死锁管理

□ 死锁的检测

(3) 分布式死锁检测



例如：场地1上的两个事务形成依赖关系 $T_1 \rightarrow T_2$ ，而 T_2 依赖于远程站点上的事务 T_3 ，判断是否处于死锁：

- 发送探测消息(1,2,3)给场地2上的事务 T_3 的管理器。
- 如果 T_3 处于非阻塞状态，则丢弃该探测消息；如果 T_3 也处于阻塞状态，在场地2上，事务 T_3 和事务 T_4 具有依赖关系 $T_3 \rightarrow T_4$ ，则探测消息更新为(1,3,4)。
- T_4 依赖于远程场地1的事务 T_1 ，因而形成探测信息(1,4,1)发送给事务 T_1 的管理器。此时， T_1 场地的事务管理器发现，探测消息中出现了回路，产生死锁。

分布式死锁管理

□ 死锁的预防与避免

T_i 是请求资源的事务
 T_j 是拥有资源的事务

(1) 死锁的预防

顺序封锁法。预先对数据对象规定一个封锁顺序，所有事务都按照这个顺序实行封锁，避免在等待图中出现回路。

(2) 死锁的解决

使用**事务的时间戳**来解决死锁，基本思路是：杀死死锁等待图中的某个事务，破坏环路。典型的算法有两种：

□ 等待-死亡(WAIT-DIE)

等待-死亡是**非抢占算法**，规则如下：

- 如果 $ts(T_i) < ts(T_j)$ ， T_i 等待；
- 否则，撤销 T_i ，并保持 T_i 原有时间戳重新启动。

等待-死亡算法倾向于新的事务，抑制老的事务。

□ 负伤-等待(wound-wait)

负伤-等待是**抢占算法**，规则如下：

- 如果 $ts(T_i) < ts(T_j)$ ，撤销 T_j ，并且将加锁的权利赋予 T_i ；
- 否则， T_i 等待。

负伤-等待规则倾向于较老的事务，但是 T_j 可以保持原来的时间戳重启。

大数据数据库并发控制技术

- 大数据环境，很多是多读少写，并且很少有复杂事务需求，如互联网环境。一般可以采用更为简单的copy-on-write技术：**单线程写，多线程读**，写的时候执行copy-on-write，写不影响读服务。
- 大数据数据库系统基于这样的假设简化了系统的设计，有些系统**摒弃了事务的概念**，减少了很多操作的代价，提高了系统性能。
- 下面介绍经典分布式数据库中的并发控制策略的**扩展策略**。

大数据库并发控制技术

□ 事务读写模式扩展

读事务模式扩展：大数据库中的读事务通常有三种模式：最新读（current read）、快照读（snapshot read）和非一致性读（inconsistent read）。最新读和快照读通常是读取单个实体组（entity group）。

- **最新读**操作会有以下保证：①一个读总是能够看到最后一个被确认的写，即满足可见性。②在一个写被确认后，所有将来的读都能够观察到这个写的结果，满足持久性。
- **快照读**，已知提交的事务时间戳，系统从那个位置直接读取数据。和最新读不同的是，此时已提交的事务的更新数据可能还没有完全生效，比如Redo日志同步成功但没有回放完成。**最新读和快照读可以保证不会读到未提交的事务。**
- **非一致性读**不考虑日志状态并且直接读取最后一个值。

大数据数据库并发控制技术

□ 事务读写模式扩展

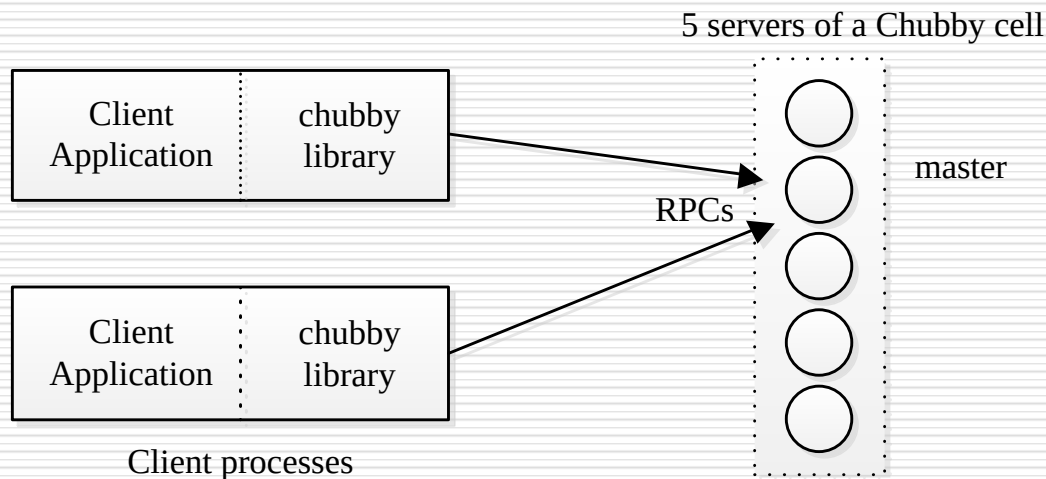
- **写事务模式**：写事务通常首先找到最新读操作，以便确定下一个可用的日志位置。提交操作将数据变更聚集到日志，并且分配一个比之前任何一个都高的时间戳，同时使用Paxos将这个log entry加入到日志中。
- **Megastore采用了读事务和写事务模式**，其完整事务生命周期包括以下步骤：
 - Step1.读：获取时间戳和最后一个提交事务的日志位置；
 - Step2.应用逻辑：从Bigtable读取并且聚集写操作到一个日志Entry；
 - Step3.提交：使用Paxos将日志Entry加到日志中；
 - Step4.生效：将数据更新到Bigtable的实体和索引中；
 - Step5.清理：删除不再需要的数据。

大数据库并发控制技术

- 封锁机制扩展：建议性锁和强制性锁
- 建议性锁(advisory lock)机制
 - 每个使用文件的进程都要主动检查该文件是否有锁存在，如果有锁存在并被排斥，则主动保证不再进行接下来的I/O操作。
 - 如果每一个进程都主动进行检查，并主动保证，那么就说这些进程都以一致性的方法处理锁。
 - 所谓一致性方法，都遵从主动检查和主动保证的处理方法。
- 强制性锁(mandatory lock)机制
 - 所有记录或文件锁功能都在内核执行。当文件被上锁来进行读写操作时，在锁定该文件的进程释放该锁之前，内核会强制阻止任何对该文件的读或写违规访问，每次读或写访问都需要检查锁是否存在。

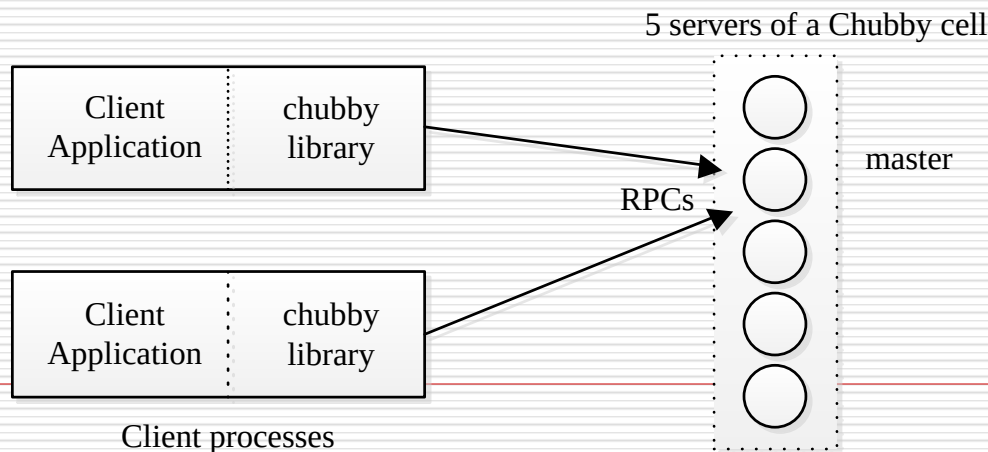
大数据库并发控制技术

- ❑ 封锁机制扩展: **Chubby**
- ❑ Chubby是Google公司研发的针对分布式系统协调管理的粗粒度锁服务, 并已应用于Bigtable系统中。
- ❑ Chubby基本架构由**客户端**和**服务器端**构成, 两者通过远程过程调用(RPC)来连接, 客户端的每个客户应用程序都有一个Chubby程序库(Chubby Library), 所有应用都是通过调用这个库中相关函数来完成。



大数据数据库并发控制技术

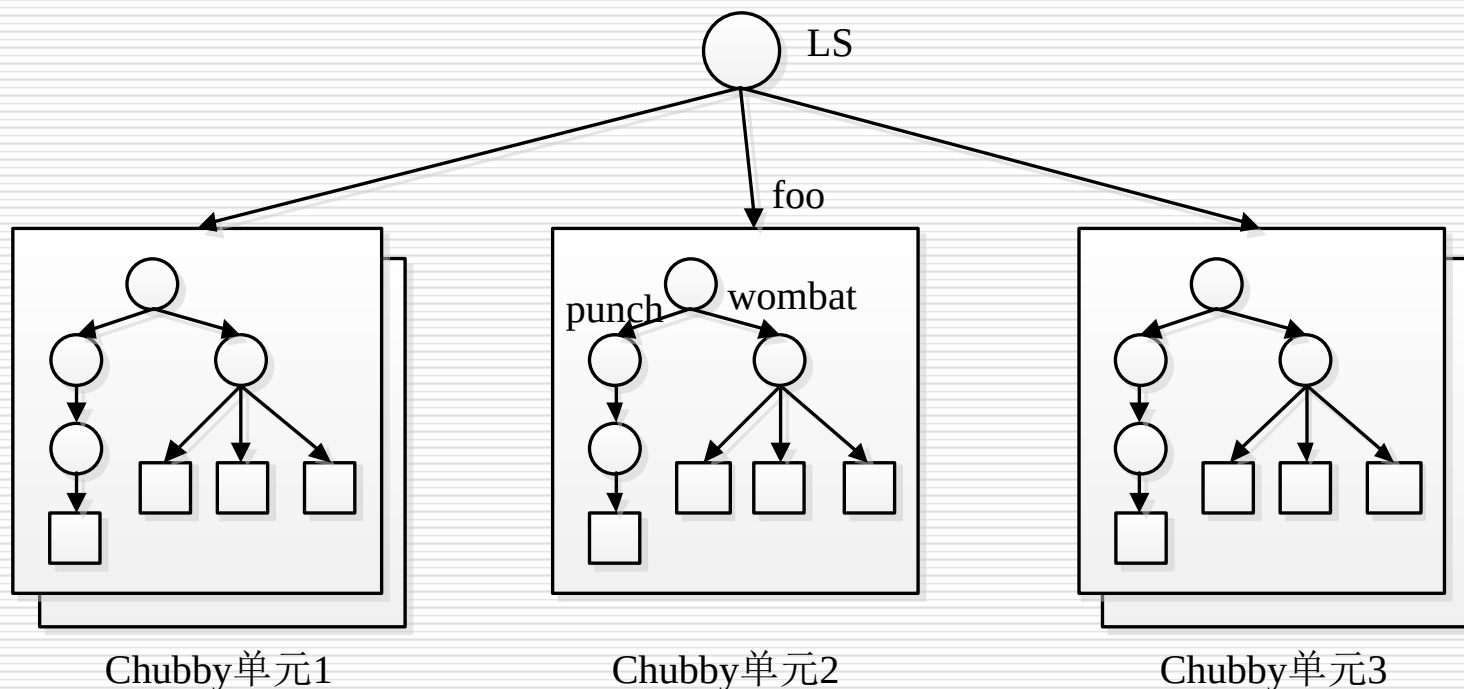
- 封锁机制扩展: **Chubby**
- 服务器端Chubby单元通常由5个副本服务器组成, 通过Paxos协议选举一台作为“**主控服务器**”, 所有读写操作都有主控服务器完成, 其他4台作为备份服务器, 在内存中维护和主控服务器一致的树形结构, 树形结构中的内容即为加锁对象或数据存储对象。



大数据数据库并发控制技术

□ 封锁机制扩展: Chubby

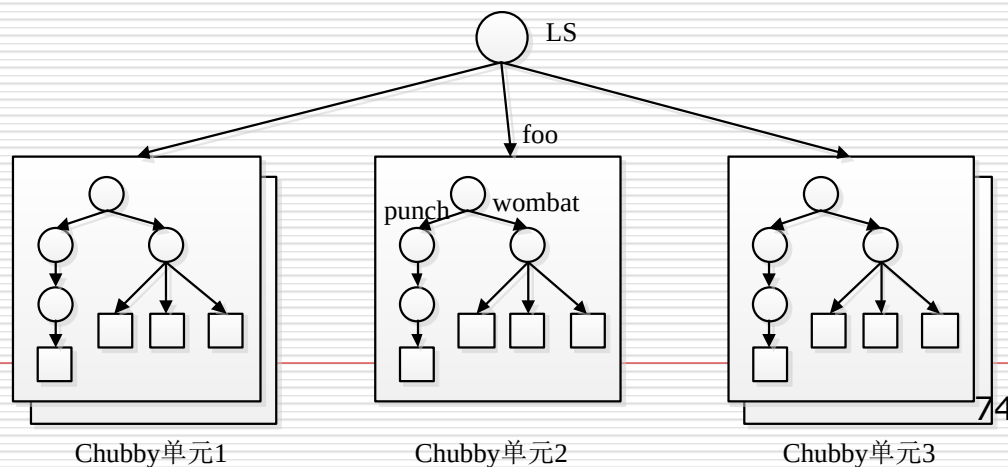
- Chubby由一系列文件和目录所组成的严格树状结构, 不同的名字单元之间通过反斜杠分割, 如: /ls/foo/wombat/pouch。



大数据数据库并发控制技术

□ 封锁机制扩展- Chubby

- 所有的Chubby单元都有一个共同前缀ls(lockservice)。第二个名字单元(foo)代表了Chubby单元的名称，通过DNS，它会被解析成一个或多个Chubby服务器。
- 一个特殊的单元名称local，用于指定使用客户端本地的Chubby单元；通常来说，这个单元都与客户端处于同一栋建筑里，因此也是最可能被访问的那个。
- 剩下的名字单元/wombat/pouch，将会由指定的那个Chubby单元自己进行解析。



大数据数据库并发控制技术

- 基于多版本并发控制扩展
 - 事务版本号分配方法
 - MVCC(Multiversion concurrency control) 的主要方法是为写事务建立版本历史记录，这个记录由一个全局唯一的事务版本号标记，并且具有递增特性。MVCC可以避免事务重启的代价，同时可以满足快照读取的要求，保证能够读到任意一个有效历史时刻上的数据。

大数据数据库并发控制技术

基本的MVCC

- 基于多版本并发控制扩展
- 事务版本号分配方法

Ts	T1	T2	A 值	A 版本号	B 值	B 版本号
0			0		0	
1	start write(A)		1	1		
2		start write(B) commit			2	2
3	write (A) commit		2	3		

- 当系统要求读取T2提交之后的快照(版本号为2的快照)，如果直接读取事务版本号小于2的数据，则A上版本号为1的修改(即A=1)也会被读取出来。但是T1尚未提交，不应该被其他事务读到中间结果，因此这种标记事务版本的方式不满足事务隔离性，需要对当前尚未结束事务的修改进行过滤，不读取未提交的事务对数据的修改结果。
- 虽然事务版本号维护比较简单，但是快照读取逻辑比较复杂，且由于事务版本号顺序没有严格反映事务的提交顺序，为保证与主机一致的事务性回放增加了处理难度。

大数据数据库并发控制技术

扩展的MVCC

- 基于多版本并发控制扩展
 - 事务版本号分配方法

Ts	T1	T2	A 值	A 版本号	B 值	B 版本号
0			0		0	
1	start write(A)		1			
2		start write(B) commit			2	2
3	write (A) commit		2	3		

- 事务提交时按照严格的事务提交顺序分配递增的事务版本号，即需要在事务提交时将分配到的事务版本号写回到本次事务修改的所有数据上。
- 对于上例，在时刻2要求进行快照读的时候，由于事务T1并未提交，事务的版本号不会小于事务T2，即不会小于2，因此不会读出A=1这个中间结果，保证了事务的隔离性。

大数据数据库并发控制技术

基本的MVCC

- 基于多版本并发控制扩展
 - 事务版本号分配方法

Ts	T1	T2	A 值	A 版本号	B 值	B 版本号
0			0		0	
1	start write(A)		1	1		
2		start write(B) commit			2	2
3	write (A) commit		2	3		

Ts	T1	T2	A 值	A 版本号	B 值	B 版本号
0			0		0	
1	start write(A)		1			
2		start write(B) commit			2	2
3	write (A) commit		2	3		

扩展的MVCC⁷⁸

大数据库并发控制技术

□ 基于时间戳并发控制扩展

Spanner使用了TrueTime方法保证事务执行过程中的时间戳的一致性。TrueTime核心思想是在不进行通信的情况下，利用高精度和可观测误差的本地时钟给事件打上时间戳，并以此比较分布式系统中两个事件的先后顺序。

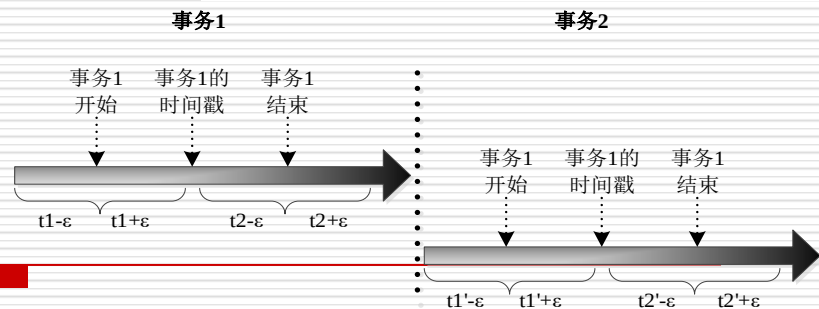
- 第一层：服务器是拥有高精度计时设备，如通常是GPS接收器和少数为防止GPS失效的原子钟。
- 第二层：所有的Spanner服务器都属于第二层，定期向多个第一层时间服务器获取时间来校正本地时钟。

大数据数据库并发控制技术

- 基于时间戳并发控制扩展
- Spanner支持的事务类型

操作	并发控制	副本要求
读写事务	悲观	Leader
只读事务	无锁	Leader时间戳，任意读
快照读，客户端提供时间戳	无锁	任意
快照读，客户端提供范围	无锁	任意

大数据数据库并发控制技术



□ 基于时间戳并发控制扩展

■ Spanner读写事务的实现

Spanner的事务会将所有的写操作先缓存起来，在Commit的时候一次提交，因此读不出在同一个事务中写的的数据。在读写事务中使用wound-wait算法来避免死锁。当客户端发起一个读写事务时，首先是读操作，先找到相关数据的leader副本，然后加上读锁，读取最近的数据。在客户端事务存活的时候会不断地向leader发心跳，防止超时。当客户端完成了所有的读操作且缓存了所有的写操作，就开始了两阶段提交。

- ✓ 客户端设置一个协调者group，给每个leader发送协调者的id和缓存的写数据；
- ✓ leader先加写锁，并寻找一个比现有事务晚的时间戳。通过Paxos记录，每一个相关的leader都要给coordinator发送他自己准备的那个时间戳。
- ✓ 协调者leader一开始也上写锁，当收到其他站点发送时间戳之后，他就选择一个提交时间戳。该提交时间戳，必须比刚刚的所有时间戳晚，而且还要比 $TT.now()+$ 误差时间还要晚。该协调者将这个信息记录到Paxos。
- ✓ 在让副本写入数据生效之前，协调者还需要等两倍时间误差，用于Paxos同步，保证任意机器上发起的下一个事务的开始时间，都晚于该事务的结束时间。
- ✓ 协调者将提交时间戳发送给客户端及副本。他们记录日志，写入生效，释放锁。

大数据库并发控制技术

□ 基于时间戳并发控制扩展

■ Spanner只读事务的实现

- ✓ Spanner首先要指定一个读事务时间戳。同时确定读操作中需要访问的所有读的Key。Spanner可以自动确定Key的范围。
- ✓ 如果Key的范围在一个Paxos group内。客户端可以发起一个只读请求给group leader。leader选一个时间戳，该时间戳要大于上一个事务的结束时间。然后读取相应数据。该事务可以满足外部一致性，读出的结果是最后一次写的结果，并且不会有不一致的数据。
- ✓ 如果Key的范围在多个Paxos group内，执行相对复杂。例如，遍历所有的group leaders，寻找最近的事务发生时间，并读取。客户端只要时间戳在TT.now().latest之后就可以满足要求。



第八章 结束

